

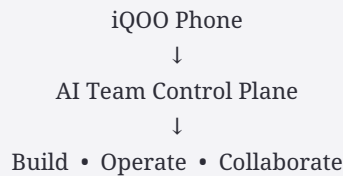
AI TEAM CONTROL PLANE

Phone-First Developer / Teams AI System

Complete Product Features, Architecture, Workflows and Technology Stack

Core idea

An AI-native engineering team that lives in your phone - developers, agents, code, infrastructure and production working together in real time.



Concept specification / Hackathon architecture

Product Idea

A phone-first AI engineering control plane where developers, team leads, AI agents, repositories, deployments, infrastructure, incidents and production systems all operate as one shared team.

The iQOO phone is the main command, notification, supervision and approval surface. The AI is not only a coding assistant; it acts as:

- coding agent
- research agent
- reviewer
- tester
- incident responder
- project coordinator
- team task router
- deployment assistant
- production monitor

Execution modes

Every task can be handled by Human / AI / Human + AI / Autonomous AI.

Complete Feature Set

1. AI coding

Users can request features or fixes directly from the phone, such as “Add dark mode to Bimax”, “Fix the checkout crash”, or “Build Google OAuth”.

- Inspect and understand the repository
- Create branches
- Edit files
- Run terminal commands
- Run tests and inspect failures
- Iterate on fixes
- Commit, push and create PRs

2. GitHub control

The phone becomes a GitHub control surface while private repositories remain protected by a GitHub App and short-lived scoped tokens.

- View repositories, branches, commits and PRs
- Inspect diffs and CI failures
- Create branches and PRs
- Approve and merge changes
- Never expose permanent GitHub credentials to the model

3. Task ownership

Every task has explicit responsibility and supervision fields.

- Requester
- Owner
- Supervisor
- Executor
- Status
- Risk

4. Human vs AI delegation

When a team lead creates a task, the system can assign it to a human, to AI, to Human + AI, or let AI choose the best owner. The assigned human can then choose manual work, AI assistance, or AI execution under supervision.

5. AI under human supervision

When a developer delegates a task to AI, the developer stays the human owner and supervisor while the AI becomes the executor.

6. AI planning before touching code

The agent analyzes the task and proposes a plan before editing. The phone can show estimated files, risk, and actions such as Approve, Modify, or Research First.

7. Context-triggered research

The coding agent can detect uncertainty and recommend current research before implementing.

- Official documentation
- Framework docs
- Relevant issues
- Best practices
- Security guidance
- Current web information

8. AI-to-AI communication

Specialized agents communicate through the orchestrator rather than one agent writing, reviewing and approving itself.

- Planner
- Researcher
- Coder
- Reviewer
- Tester
- Deployment Agent

9. Intelligent diff explanation

The phone explains what a diff means instead of showing raw patches only. It can summarize files changed, behavior added, risks, and suggested verification steps.

10. Conversational code review

Developers can ask questions such as “Why did you modify settings.ts?” or request alternate implementations. The AI reassesses the code and explains tradeoffs.

11. AI reviewer

A separate reviewer checks architecture, bugs, security, duplicate code, dependencies, edge cases, performance and regression risk.

12. Testing agent

A separate tester can run unit, integration, type-checking, lint, build, browser, UI and API tests and return evidence.

13. Evidence-based completion

The system should never simply say “Done”. It should report code evidence, test evidence, build evidence, research evidence, runtime evidence and confidence.

14. Automatic developer ownership

For incidents or new tasks, AI can infer the best owner from CODEOWNERS, git history, PRs, role, expertise, workload and availability.

15. Developer roles

Workspaces can include Frontend, Backend, AI/ML, DevOps, QA, Designer, Security and Team Lead roles while also learning real ownership from code history.

16. Team accounts

All developers join the same workspace and share projects, tasks, agents and engineering context.

17. Live presence

The system shows who is available and what each teammate is actively doing. Updates propagate in real time.

18. Live task awareness

Everyone can see task owner, active AI agent, current stage, branch and status.

19. Agent-aware collaboration

AI detects when a human or another agent is already modifying the same component and can wait, coordinate, or ask before editing.

20. Automatic task handoff

When one task exposes a dependency in another specialty, the system suggests the appropriate teammate or AI agent and hands the task over.

21. Dependency graph

The system models dependencies between frontend, backend, database, environment configuration, tests and deployment, then automatically unblocks downstream work.

22. AI project manager

A high-level request like “Ship Google login today” can be decomposed into frontend, backend, schema, QA, DevOps and deployment tasks and tracked automatically.

23. Human work with AI shadowing

If a developer chooses manual work, AI can still watch the branch, commits, tests, PR and build and flag omissions.

- Manual
- AI Assisted
- AI Executed
- Autonomous

24. Team memory

The system stores team decisions and engineering rules, such as avoiding new dependencies, and warns agents when proposals violate them.

25. Smart escalation

Repeated agent failures can trigger research, a stronger model, a specialist teammate or the human owner instead of endless retry loops.

26. Model escalation

Tasks can be routed by complexity and modality.

- Phone local model - intent
- Cheap cloud model - summarization
- Coding model - implementation
- Powerful reasoning model - hard architecture
- Vision model - screenshots
- Research model - web investigation

27. On-device AI

A small model on the iQOO handles voice intent, command classification, summarization, notification prioritization, local suggestions, routing and simple reasoning.

28. Cloud AI

Large models handle coding, architecture, difficult debugging, research synthesis, review and reasoning. NVIDIA NIM or another provider can supply these models.

29. Cloud computer

Agents receive an isolated execution environment, such as Vercel Sandbox, with Linux filesystem, Git, Node, Python, terminal, dependencies, dev server and tests.

30. Cloud coding agent

The coding agent is a strong LLM + tool loop + sandbox + Git + GitHub App. The model asks for tool actions; the runtime executes them in the sandbox.

31. Private repository support

A GitHub App creates a short-lived installation token; the sandbox uses it to clone the private repo. Permanent credentials never reach the phone or model.

32. Automatic feature shipping

A request like “Ship dark mode” can execute Plan → Research → Branch → Code → Tests → Review → PR → Human Approval → Merge → Build → Deploy → Verify.

33. Deployment monitoring

The system tracks queued, building, deploying, live, failed and rolled-back states and presents production health on the phone.

34. Vercel integration

For web projects, GitHub can feed Vercel previews and production deployments; the phone can show preview URLs, build failures, logs and production status.

35. Native desktop release support

Desktop apps such as Bimax use a platform-native CI runner after coding. For macOS: cloud coding sandbox → GitHub → macOS CI runner → .app → .dmg.

36. Platform-specific build workers

The orchestrator selects a builder based on target platform.

- Web → Linux
- macOS → macOS runner
- Windows → Windows runner
- Android → Android runner

37. DMG release pipeline

For Bimax: AI edits code → GitHub PR → macOS runner → build app → package DMG → sign/notarize → upload release.

38. Release storage

Websites can stay on Vercel while binaries such as Bimax.dmg live in Cloudflare R2, AWS S3, GitHub Releases or Vercel Blob.

39. Production verification

A successful deployment is not enough. Agents verify the site loads, downloads work, checksums match, apps launch and the new feature behaves correctly.

40. Production monitoring

Connected services feed runtime errors, exceptions, HTTP failures, latency, deploy failures, crashes, CPU, memory and database problems.

41. AI incident detection

The system correlates incidents with deployments, logs, recent commits, databases, APIs, Cloudflare and Vercel to identify probable regressions.

42. Incident ownership

AI maps the failing code or service to the likely developer and sends the responsible person a phone notification.

43. AI incident investigator

From the phone, an engineer can trigger investigation that collects logs, finds the failing endpoint, traces source code and commits, reproduces the issue and suggests a fix.

44. Rollback

High-risk rollback can require phone approval, then the control plane invokes the provider or deployment system to restore a known-good version.

45. Safe autonomy

Actions are categorized by risk.

- LOW: read logs, search code, run tests, research - automatic
- MEDIUM: modify code, branch, PR - supervised
- HIGH: merge, production deploy, DB migration, DNS, secrets, rollback, billing - human approval

46. Phone as security root

Cloud AI can reason and agents can work, but dangerous actions require trusted phone authorization, optionally with biometrics.

47. Smart notifications

Notifications are actionable and contextual: task name, changed files, tests, review concerns, risk and buttons such as Review, Ask AI and Approve.

48. Live logs

The phone can expose application, deployment, Cloudflare, AWS, CI and agent logs while AI summarizes thousands of lines into the likely root cause.

49. Infrastructure integrations

Connectors can include GitHub, Vercel, Cloudflare, AWS, Supabase, CI/CD, monitoring and billing providers.

50. No universal master key

Each provider remains independently authenticated with least-privilege scopes. Agents receive temporary capabilities rather than raw secrets.

51. Cost monitoring

The phone can aggregate AWS, Vercel, NVIDIA AI, Cloudflare and other costs and detect abnormal increases.

52. AI cost routing

The model router uses cheap models for easy work and stronger models for hard tasks, balancing quality, latency and cost.

53. Unified developer dashboard

GitHub, Vercel, Cloudflare, AWS, Supabase, monitoring and CI information converge in the iQOO app.

54. Web research agent

Research is connected directly to engineering tasks and can investigate documentation, framework changes, security information, errors, APIs and implementation techniques.

55. Cloud browser / cloud computer

Agents can use a cloud browser for deployed-site testing, documentation research, browser bug reproduction and visual verification.

56. Camera-to-engineering

The iQOO camera can capture errors, broken UIs, whiteboards or designs and connect that visual evidence back to the codebase.

57. Screenshot-to-bug

A screenshot can be analyzed to identify the UI, locate the likely component/CSS, create an issue and offer an automated fix.

58. Voice DevOps

Developers can ask “How is production?” and receive summarized health, deployments, errors and latency, then ask follow-up questions such as “Why?”.

59. Shared AI awareness

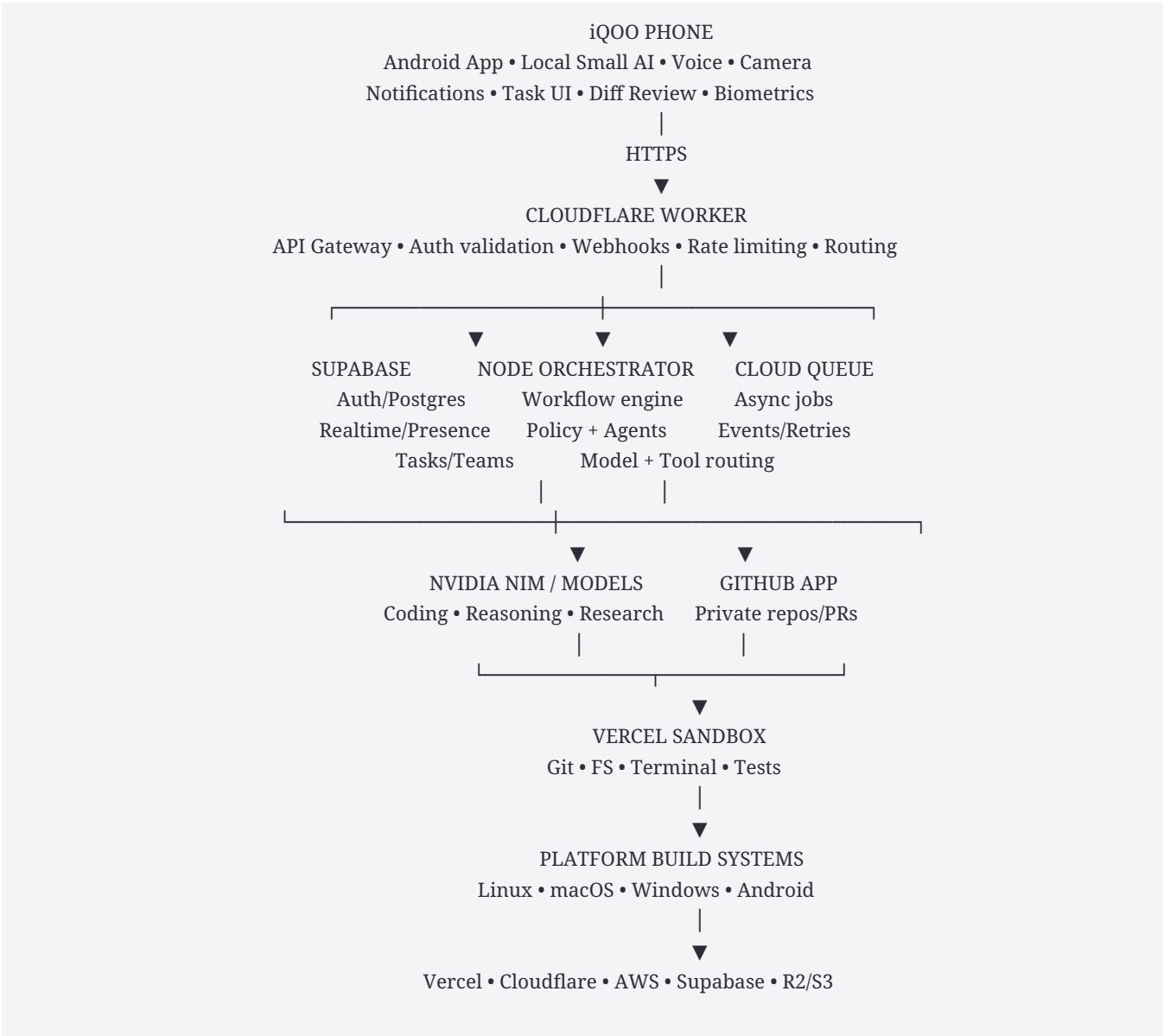
A personal phone agent knows the developer’s own tasks and responsibilities while a team agent knows the full project, team, incidents, deployments, dependencies and ownership.

60. Personal agents, team coordinator and AI meetings without meetings

Each developer can have a personalized agent while the team coordinator routes tasks, detects blockers, manages load and produces team briefs without manual status meetings.

Complete System Architecture

The following architecture shows where each component physically lives and what responsibility it owns.



What exactly is the control plane?

Control plane
Cloudflare Worker + Node.js Orchestrator + Supabase. Together they form the product control plane, but each component has a different responsibility.

Cloudflare Worker

- Lives on Cloudflare
- Public API for the iQOO app
- Receives provider webhooks
- Request routing
- Rate limiting
- Authentication validation at the edge

Node.js Orchestrator

Hosted on a service suitable for longer-running backend workloads, such as Google Cloud Run, Railway, Render, Fly.io or AWS.

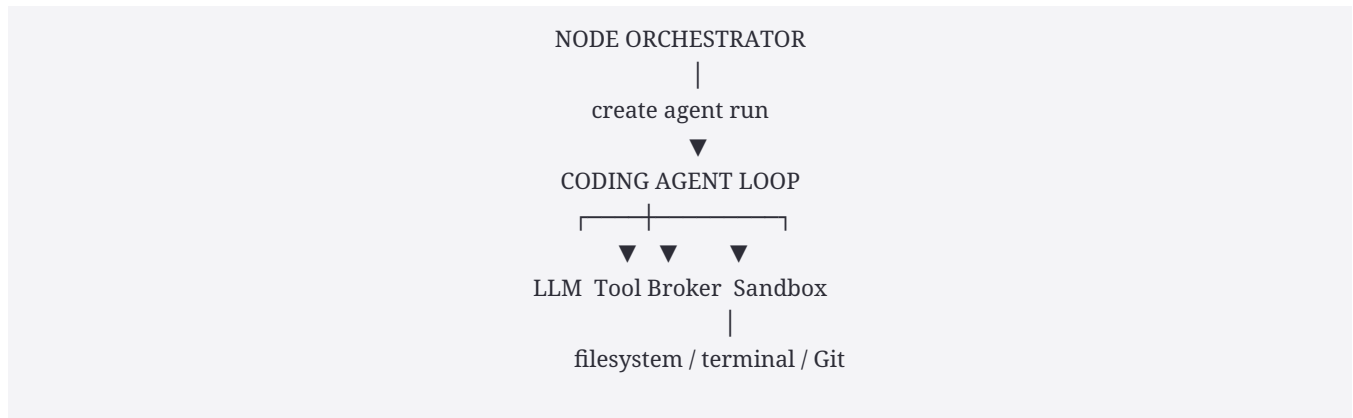
- AI workflows
- Agent creation
- Tool calling
- Task delegation
- Research workflows
- GitHub actions
- Sandbox creation
- Model calls
- Approval policy
- Incident logic
- Ownership engine

Supabase

Supabase stores shared, durable team and product state and provides Auth, Postgres, Realtime and Presence.

- users
- teams
- projects
- roles
- tasks
- task assignments
- agent runs
- incidents
- approvals
- deployments
- events
- messages
- presence

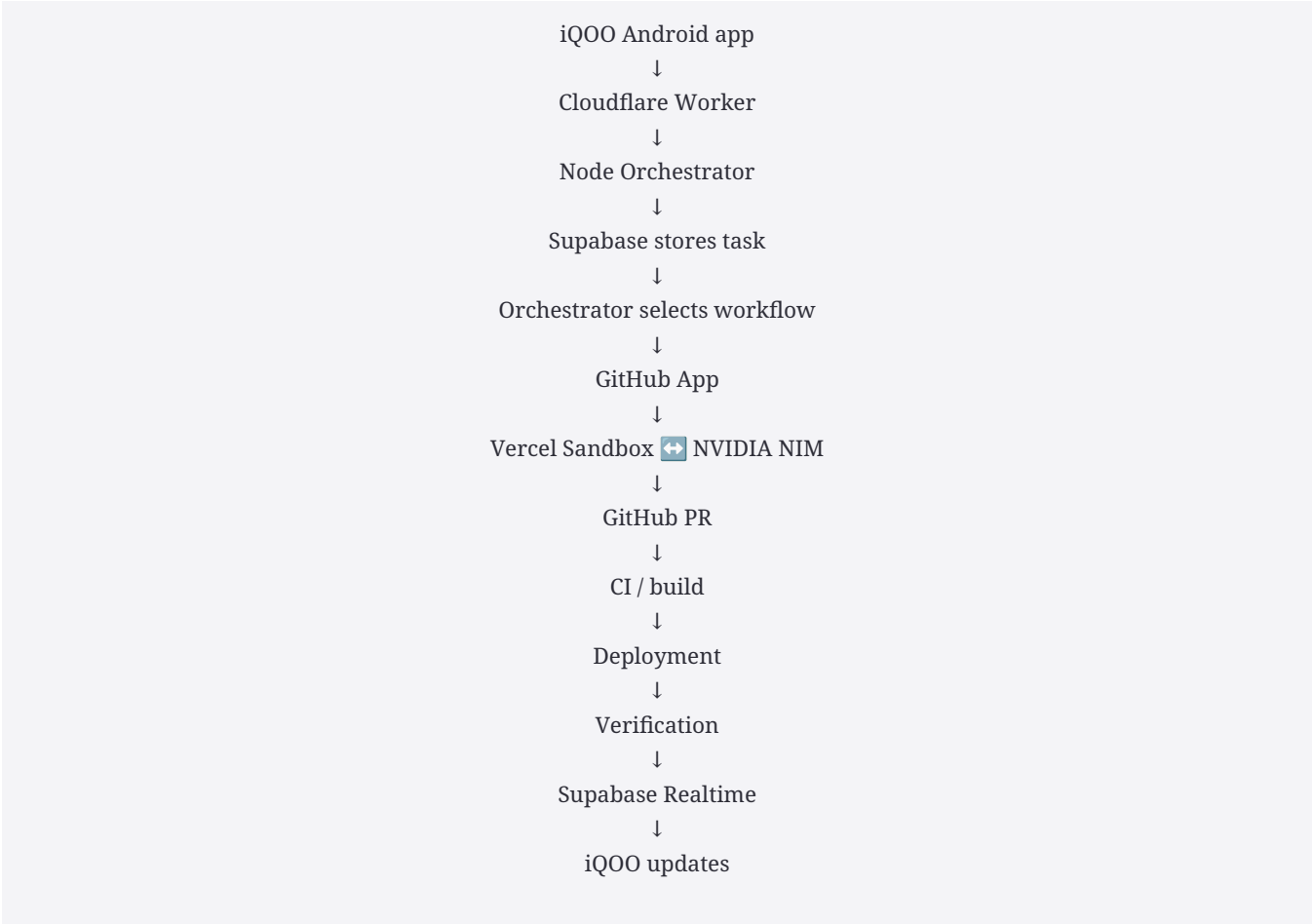
Cloud Agent Architecture



Example: the model asks to search for ThemeProvider. The tool broker converts that intention into `search_code("ThemeProvider")`, the sandbox executes it, and the result is returned to the model. This repeats until the task is complete.

Core End-to-End Flows

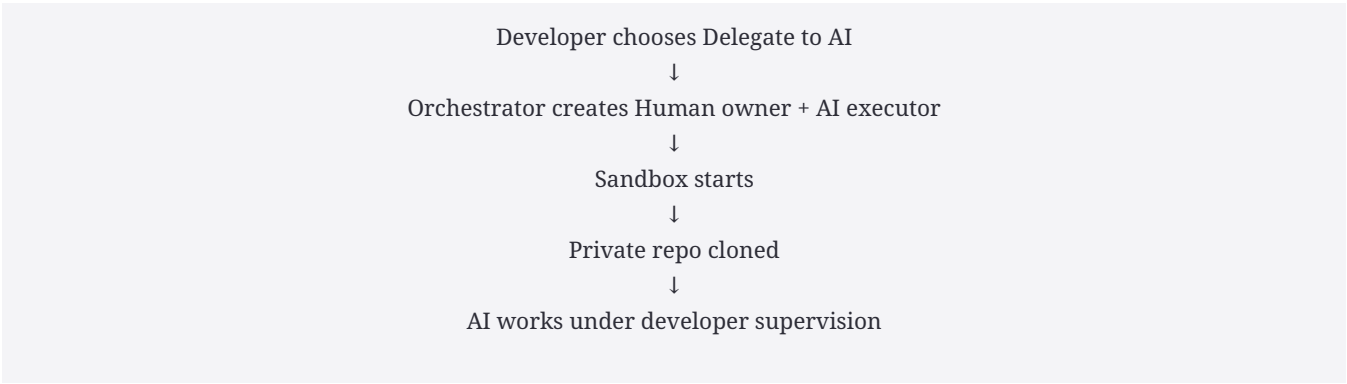
Phone request architecture



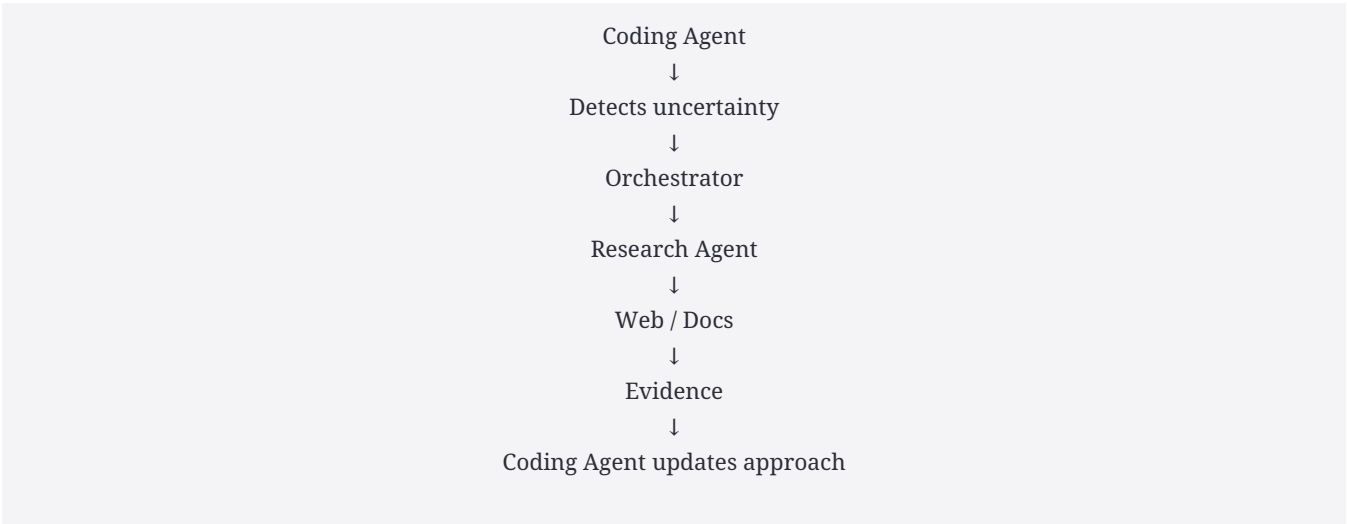
Team delegation architecture



Delegated AI architecture



Research loop



Review loop



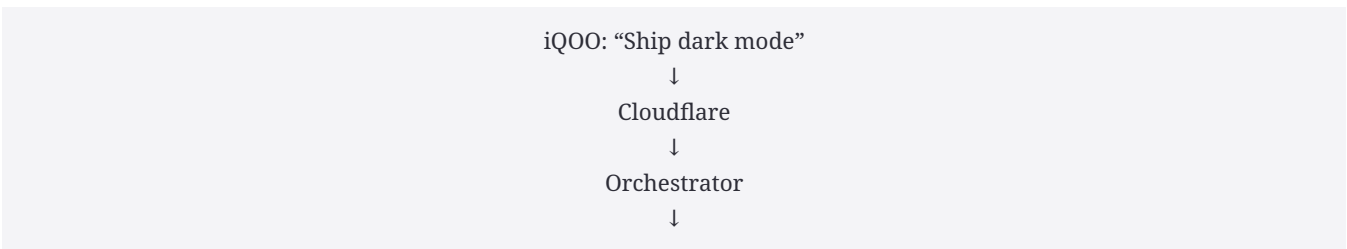
Incident loop



Production deployment loop



Bimax-specific flow





Recommended Concrete Technology Stack

Layer	Recommended technology
Phone	Android, Kotlin, Jetpack Compose, small local model, Firebase Cloud Messaging
Edge	Cloudflare Workers, Cloudflare Queues
Database / Realtime	Supabase Postgres, Auth, Realtime, Presence
Orchestrator	Node.js + TypeScript on Cloud Run or Railway
AI	NVIDIA NIM + routing / research / vision models
Agent execution	Vercel Sandbox
Source control	GitHub App + GitHub Webhooks
Web deployment	Vercel
Desktop builds	GitHub Actions macOS runners
Infrastructure integrations	AWS APIs, Cloudflare APIs, Vercel APIs
Binary storage	Cloudflare R2 / AWS S3
Push notifications	Firebase Cloud Messaging

Product Hierarchy



Positioning

An AI-native engineering team that lives in your phone - developers, agents, code, infrastructure and production working together in real time.

The novel part is not merely that AI can code. The system understands the team, delegates work, operates under human supervision, researches uncertainty, explains changes, gets reviewed by

other agents, verifies results, manages production and asks the correct human for approval when needed.

Final Product Thesis

This product is not simply an AI coding assistant on a phone. It is a supervised human-AI engineering operating layer that connects team ownership, code execution, research, review, testing, infrastructure, deployment, monitoring and approval into one mobile control surface.

- Humans keep ownership and approval authority while AI performs delegated work.
- Specialized agents research, code, review, test and operate production through controlled tools.
- The iQOO becomes the trusted command, collaboration, notification and approval device for the engineering team.